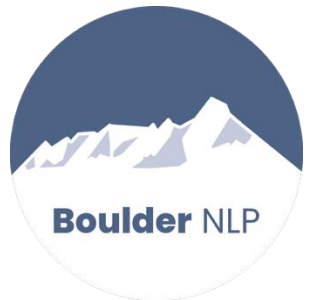


Natural Language Processing

Jim Martin -- Lecture 4
CSCI 5832



Today

Probabilistic Language Modeling (Chapter 3)

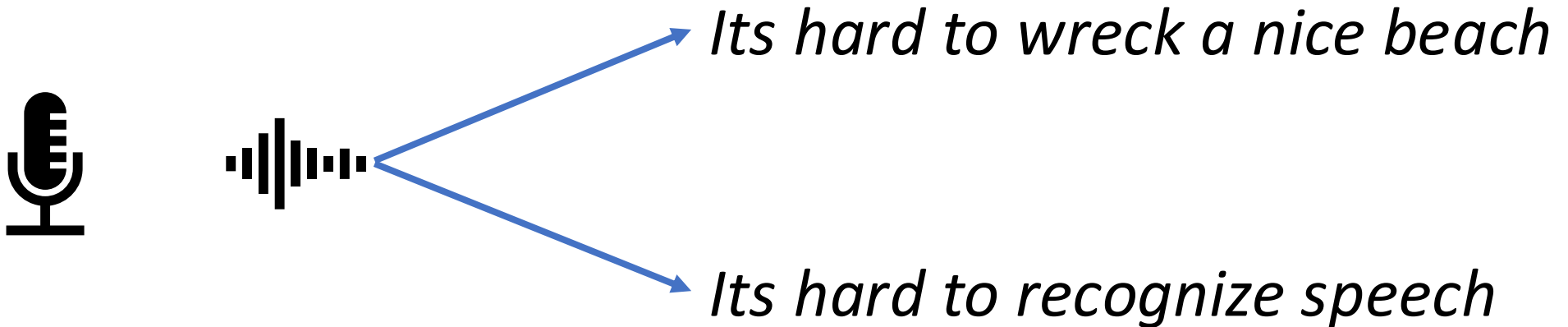
Applications

The ability to assess the probability of a sequence is core to almost every modern NLP application (independent of LLMs). If you've used any of these applications, you've interacted with a language model.

- Autocomplete
 - Texting, composing emails, etc.
- Automatic speech recognition
- Handwriting and character recognition
 - Optical character recognition (OCR)
- Email spam detection
- Sentiment analysis
- Spelling correction
- Machine translation
- Summarization
- Etc.

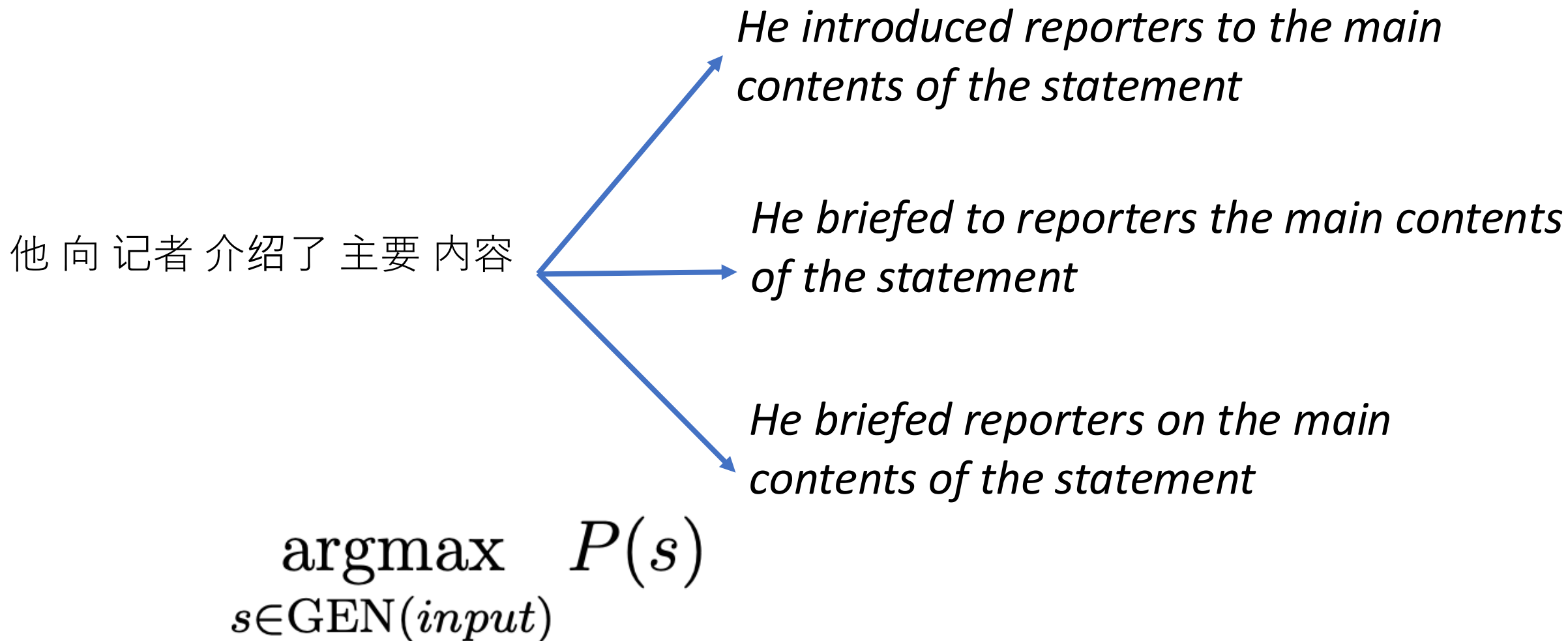
Speech Recognition

- An (acoustic) hypothesis generation system proposes multiple possible sequences for an input signal...



$$\operatorname{argmax}_{s \in \text{GEN}(\text{input})} P(s)$$

Machine Translation



Sentiment Analysis

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life.

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life. In short, our stay was fantastic.

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life. In short, our stay was horrible.

$$\operatorname{argmax}_{s \in \text{GEN}(\text{input})} P(s)$$

Contextual Generation

These applications make use of a language model's ability to assign a probability to a text and then use that text to score various options.

$$P(w_{1:n})$$

But where do those options come from? It turns out we can use language models for the generator part as well, by exploiting their ability to predict the next word.

- Given some context,
 1. Generate a distribution over the possible next words.
 2. Randomly pick a word from that distribution respecting the various probabilities.
 3. Add that word to the context and iterate.

$$P(w_n | w_{1:n-1})$$

Contextual (Autoregressive) Generation

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life. In short, our stay was _____

$$P(w_n | w_{1:n-1})$$

Contextual (Autoregressive) Generation

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life. In short, our stay was uncomfortable

$$P(w_n | w_{1:n-1})$$

Contextual (Autoregressive) Generation

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life. In short, our stay was uncomfortable and

$$P(w_n | w_{1:n-1})$$

Contextual (Autoregressive) Generation

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life. In short, our stay was uncomfortable and not worth the price we paid. We will not be returning to this hotel.

$$P(w_n | w_{1:n-1})$$

A Generative Story

So why does that approach work?

Let's return to the notion of the probability of a sentence. We clearly can't rely on our usual frequentist intuitions to understand this concept.

But reformulating the problem as a word prediction task combined with the Chain Rule for probability provides us with an alternative way to view sequence probabilities.

$$P(w_{1:n})$$

$$P(w_n | w_{1:n-1})$$

The Chain Rule

- Using the definition of conditional probabilities along with some rewriting
- And thinking of sequences as a kind of joint event...

$$P(B|A) = \frac{P(A, B)}{P(A)}$$
$$P(A, B) = P(A)P(B|A)$$

$$P(x_1, x_2, \dots, x_n) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2) \dots, P(x_n|x_1 \dots x_{n-1})$$

- Leads us to the following notation we'll see again and again

$$P(w_{1:n}) = P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \dots, P(w_n|w_1 \dots w_{n-1})$$
$$= \prod_{i=1}^n P(w_i|w_{1:i-1})$$

The Chain Rule

How does that look with a real example?

$$P(w_{1:n}) = P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \dots, P(w_n|w_1 \dots w_{1:n-1})$$

P(its water is so transparent)=

P(its)*

P(water | its)*

P(is | its water)*

P(so | its water is)*

P(transparent | its water is so)

Consider each word in the sentence incrementally, conditioning each on the previous words.

The probability of the resulting sequence is the product of each choice in the chain.

How does that Chain Rule Help?

Let's consider the components near the beginning of the chain.

$$P(\text{its})$$

$$P(\text{water}|\text{its})$$

$$P(\text{was}|\text{its, water})$$

$$P(\text{its water was so transparent}) =$$

$$P(\text{its})^*$$

$$P(\text{water}|\text{its})^*$$

$$P(\text{was}|\text{its water})^*$$

$$P(\text{so}|\text{its water was})^*$$

$$P(\text{transparent}|\text{its water was so})$$

$$P(w_n | w_{1:n-1})$$

Candidate Models

Over the rest of the course, we'll see 4 ways of implementing language models, ultimately leading to transformers.

1. *N*-gram models.
 - Models based on counts from a corpus.
2. Feedforward neural models
3. Recurrent NNs
4. Transformers



N-Gram Language Models

N-Gram language models are a method for computing the probability of sequences using:

- Counts of sequences of length *N* gathered from a corpus.
- Along with a simple independence assumption applied to the chain rule view of sequence probabilities.

$$P(s)$$

$$P(w_{1:n})$$

An Example of Statistical Investigation of the Text *Eugene Onegin* Concerning the Connection of Samples in Chains

N-Gram

A. A. Markov

(Lecture at the physical-mathematical faculty, Royal Academy of Sciences, St. Petersburg, 23 January 1913)¹

N-Gram language
computing the
using:

- Short
- Count
- Along with
assumption
view of

This study investigates a text excerpt containing 20,000 Russian letters of the alphabet, excluding **Ъ** and **Ь**,² from Pushkin's novel *Eugene Onegin* – the entire first chapter and sixteen stanzas of the second.

Prediction and Entropy of Printed English

By C. E. SHANNON

(Manuscript Received Sept. 15, 1950)

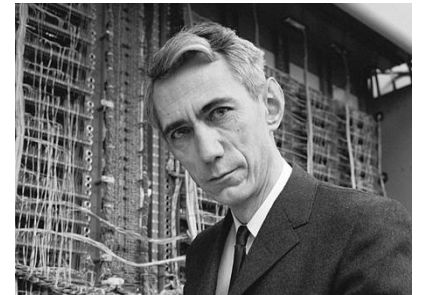
A new method of estimating the entropy and redundancy of a language is described. This method exploits the knowledge of the language statistics possessed by those who speak the language, and depends on experimental results in prediction of the next letter when the preceding text is known. Results of experiments in prediction are given, and some properties of an ideal predictor are developed.

1. INTRODUCTION

$P(w_{1:n})$ = **I**N A previous paper¹ the entropy and redundancy of a language have been defined. The entropy is a statistical parameter which measures, in a certain sense, how much information is produced on the average for each letter of a text in the language. If the language is translated into binary



A. A. Markov (1856)



The Chain Rule

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{1:i-1})$$

$$P(w_{1:n}) = P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \dots, P(w_n|w_1 \dots w_{1:n-1})$$

P(The water is so transparent)=

P(The)*

P(water | The)*

P(is | The water)*

P(so | The water is)*

P(transparent | The water is so)

Consider each word in the sentence incrementally, conditioning each on the previous words.

The probability of the resulting sequence is the product of each item in the chain.

N-Gram Language Modeling

How do we acquire the probabilities we need to instantiate this factored model?

- Start with the definition of conditional probabilities
- and then look for counts from a corpus to support them.

$$P(w_i | w_{1:i-1}) = \frac{P(w_{1:i})}{P(w_{1:i-1})} = \frac{\frac{\text{Count}(w_{1:i})}{M}}{\frac{\text{Count}(w_{1:i-1})}{M}} = \frac{\text{Count}(w_{1:i})}{\text{Count}(w_{1:i-1})}$$

Examples

Let's consider the case of bigrams... sequences of length 2.

- Like “the water”.

If we're seeking is $P(\text{water} | \text{the})$. That's just

$$P(\text{water} | \text{The}) = \frac{\text{Count}(\text{The, water})}{\text{Count}(\text{The})}$$

What about trigrams?

$$P(\text{is} | \text{The, water})$$

Examples

What about 7-grams?

$$P(\text{the}|\text{The, water, is, so, transparent, that}) = \frac{\text{Count}(\text{The, water, is, so, transparent, that, the})}{\text{Count}(\text{The, water, is, so, transparent, that})}$$

Using the InfiniGram online N-gram server those counts are 49 and 192 giving us roughly 0.25 for "the" as the next word!

N -Gram Counts

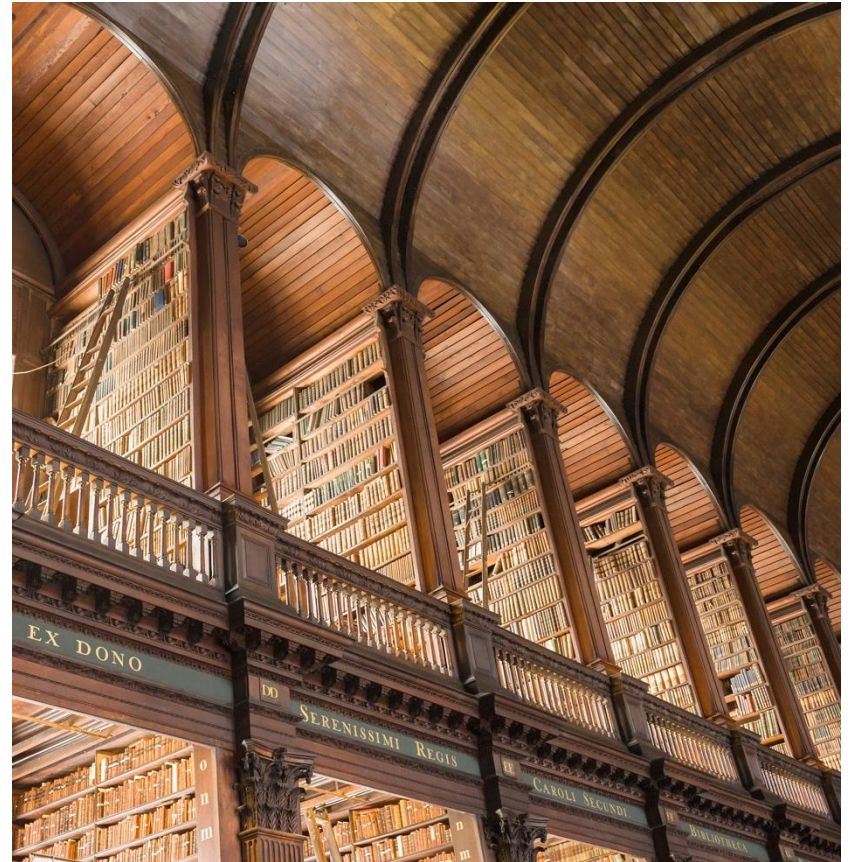
- For most longer sequences, and for most text collections, we won't get good estimates using simple counting.
 - We'll get a lot of 0 counts, leading to 0 probabilities for sequences that are entirely plausible.
- Clearly, we'll have to be more clever to make this counting approach work.

N -Gram Counts and Corpora

To see the difficulty of our task, consider how many N -grams are possible given a fixed vocabulary of size $|V|$, say on the order of 30k.

For bigrams, there are 900M possibilities. For trigrams, its 27 Trillion!

With a corpus of 1M words, we couldn't possibly come up with counts for all but a small fraction of the bigrams.



Independence Assumption

$$P(w_{1:n}) = P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \dots, P(w_n|w_1 \dots w_{1:n-1})$$

Let's assume the probability of the next word is *independent* of some of its earlier history. So...

Bigram

- $P(\text{transparent} | \text{The, water, is, so}) = P(\text{transparent} | \text{so})$

Or perhaps

Trigram

- $P(\text{transparent} | \text{The, water, is, so}) = P(\text{transparent} | \text{is, so})$

The probability is dependent only on a history of a fixed length.



A. A. Mapson (1886).

Markov Assumption

Replace every component in the chain rule product with an approximation using a fixed size immediately preceding history.

Bigram version

$$P(w_i | w_{1:i-1}) \approx P(w_i | w_{i-1})$$

Trigram

$$P(w_i | w_{1:i-1}) \approx P(w_i | w_{i-2:i-1})$$

General case

$$P(w_i | w_{1:i-1}) \approx P(w_i | w_{i-(N-1):i-1})$$

Chain Rule+Markov Assumption

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{1:i-1})$$

$$P(w_{1:n}) \approx \prod_{i=1}^n P(w_i | w_{i-(N-1):i-1})$$

N = 2

P(the water is so transparent)=

P(the)*

P(water | the)*

P(is | water)*

P(so | is)*

P(transparent | so)

N=3

P(the water is so transparent)=

P(the)*

P(water | the)*

P(is | the, water)*

P(so | water, is)*

P(transparent | is, so)



Berkeley Restaurant Project

- *Can you tell me about any good cantonese restaurants close by*
- *Mid priced thai food is what I'm looking for*
- *Tell me about chez panisse*
- *Can you give me a listing of the kinds of food that are available*
- *I'm looking for a good place to eat breakfast*
- *When is caffe venezia open during the day*

Bigram Counts

- Vocabulary size is 1446 $|V|$
- 9222 sentences
 - “I want” occurred 827 times

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Bigram Probabilities

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

- Divide bigram counts by the prefix unigram counts to get bigram probabilities.

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.00033	0.00033	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

$$\begin{aligned} P(\text{want} \mid \text{i}) &= 827/2533 \\ &= .336 \end{aligned}$$

Bigram Estimates of Sentence Probabilities

$$\begin{aligned} P(< s> \text{ I want english food } < /s>) = \\ & P(\text{I} | < s>) * \\ & P(\text{want} | \text{I}) * \\ & P(\text{english} | \text{want}) * \\ & P(\text{food} | \text{english}) * \\ & P(< /s> | \text{food}) * \\ & = .000031 \end{aligned}$$

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{1:i-1})$$

Implicit Knowledge in Language Models

N-gram language models capture a surprising range of interesting facts about language.

- $P(\text{english} | \text{want}) = .0011$

- $P(\text{chinese} | \text{want}) = .0065$

- $P(\text{to} | \text{want}) = .66$

- $P(\text{eat} | \text{to}) = .28$

- $P(\text{want} | \text{spend}) = 0$

- $P(I | \langle s \rangle) = .25$

- $P(I | I)$

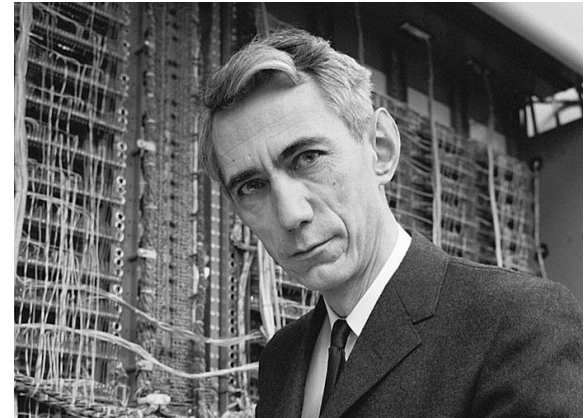
World knowledge

Syntax

Dialog

Autoregressive Generation

An illuminating (and very useful) way to employ a language model is to use it to generate random sentences that reflect the statistics of the data from which the model was derived. This idea was proposed by Claude Shannon in the 1950s as part of his work on Information Theory and especially the notion of Entropy.



Autoregressive Generation

- Sample a random bigram ($\langle s \rangle, w_i$) according to the model's probability distribution over bigrams.
- Next sample a new bigram (w_i, x) where the prefix w_i matches the suffix of the previously sampled bigram.
 - And so on until we randomly sample some ($w_i, \langle /s \rangle$).
 - String them all together and you have a sentence. Do that repeatedly and you have a corpus.

$\langle s \rangle$ |

I want

want to

to eat

eat Chinese

Chinese food

food $\langle /s \rangle$

what about thai food

i'd like to go on saturday

where's the best vegetarian restaurant for this friday

i'm willing to go to a nice romantic restaurant that
allows smoking

it should be less than forty minutes

american food i eat on a sunday

the cost of a mile from here

i have a car so i could travel anywhere in berkeley

is there a restaurant that serves only vegetarian
foods

i want to hear more about kirala

maybe we can get a cheeseburger and beer

um show me the list again



Autoregressive Generation

- We can also compare the outputs from various orders of N -gram models. Here's some output from a Shakespeare generator.

Unigram

To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have
Every enter now coverally so let

Bigram

Hill. What means, sir. I confess she? then all sorts, he is trim, captain.

Trigram

Sweet prince, Falstaff shall die. Harry of Monmouth's grave.
This shall forbid it should be branded, if renown made it empty.
Indeed the duke: and had a verv good friend.

ollow.
an?

Quadrigram

Fl.
King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;
Will you not tell me who I am?
It cannot be but so.

Shakespeare

- $N=884,647$ tokens, $V=29,066$
- Shakespeare produced 300,000 bigram types out of $V^2= 844$ million possible bigrams...
 - So, 99.96% of the possible bigrams were never seen (have zero entries in the table)
 - This is the biggest problem in N-gram language modeling.
- 4-grams are worse: What's coming out looks like Shakespeare because *it is Shakespeare.*

Recap

- Count-based N -grams are a widely-used, lightweight method to implement probabilistic language models.
- Models that can compute probabilities of sequences, or conditional probabilities of next words given a history.
- Higher-order N -grams suffer from sparsity but improve performance.
 - We'll approach sparsity in the next class and how to assess performance in the following one.

Smoothing

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{1:i-1})$$

Let's go back to Shakespeare! He produced 300K bigram types out of a possible $V^2 = 844$ million bigrams, meaning 99.96% of the bigrams that were possible given his vocabulary were never observed!!

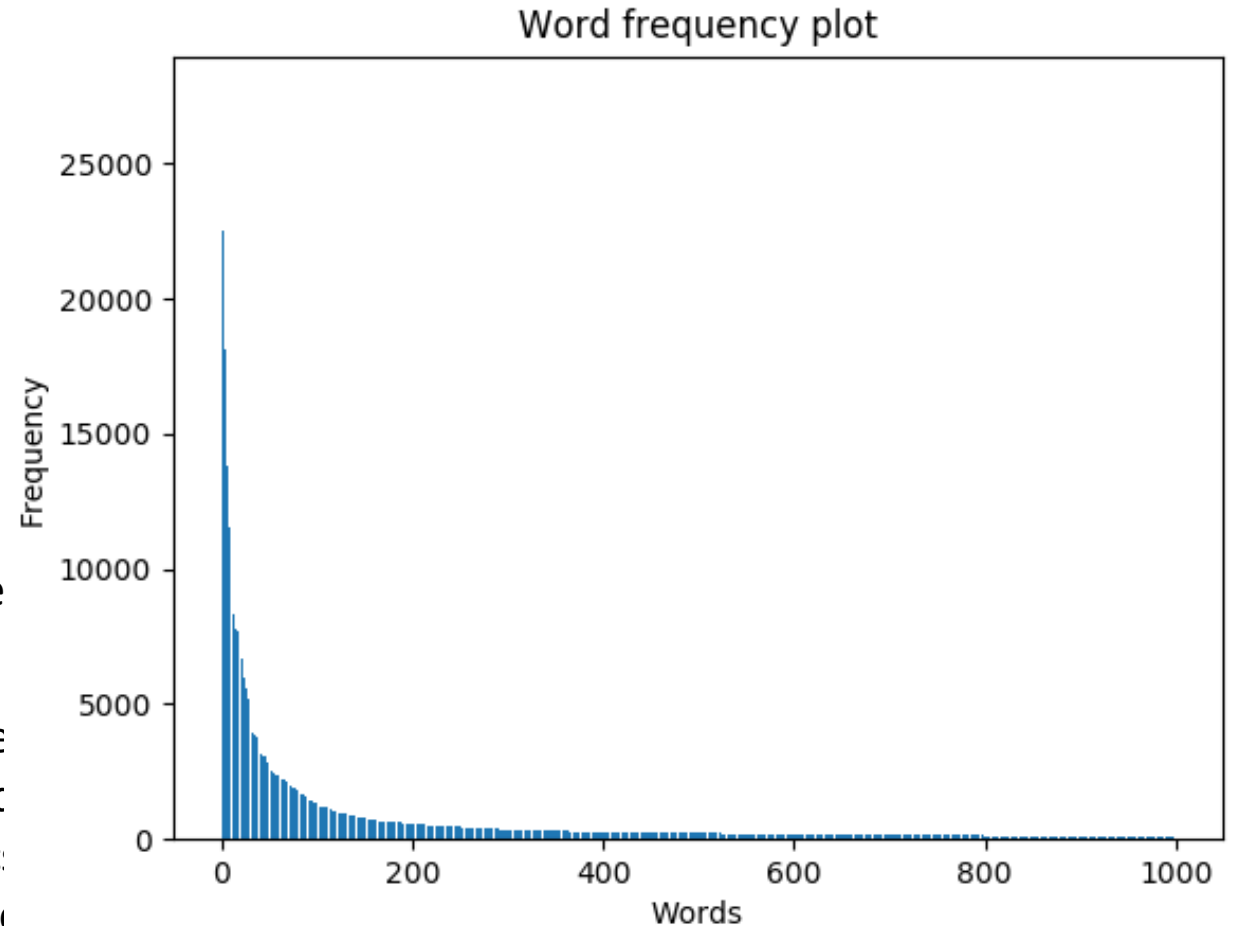
This is unsurprising since Shakespeare's entire corpus is only 840k words. That's an upper bound on the total number of bigrams he could have produced!

Consequences of this sparsity:

- In assessing the probability of test sentences, any sentence containing one of those unseen bigrams will have a probability of 0!
- And for sampling-based generation, it means we will never emit sentences containing those bigrams --- severely limiting the diversity of any output.

Zero Counts

- Some of those zeros are really zeros...
 - Events that shouldn't ever happen
 - On the other hand, some of them are just corpus had been a little bigger they would
 - What would that count be in all like
- Zipf's Law (long tail phenomenon):
 - A small number of events occur with high frequency
 - A large number of events occur with low frequency
 - With a corpus we can quickly collect statistics
 - But we might have to wait a very long time to see rare events
- Result
 - Our estimates are sparse! We have no counts at all for the vast number of things we want to estimate!



Overcoming Sparsity in N-gram Models

Sparsity doesn't only come in the form of zero-counts. The estimates for small-count N -grams are likely to be poor as well. To overcome sparsity, we'll use smoothing, which is just a cover term for a bag of related methods that try to estimate counts for things that haven't happened (or happened rarely).

- Additive smoothing
- Discounting
 - Steal from the rich, re-distribute the wealth
- Backoff
 - Avoid zero counts by resorting to shorter N -grams
- Interpolation
 - Combining information from N -gram models of different lengths

Additive Smoothing

Add a little bit to all the counts and the zeroes go away!

- If we +1, it's called Add-1 smoothing.

For unigrams:

$$P(w) = \frac{\text{Count}(w)}{M}$$

$$P_{+1}(w) = \frac{\text{Count}(w) + 1}{M + V}$$

Additive Smoothing

Add a bit to all the counts and the zeroes go away!

- If we add 1, it's called Add-1 smoothing.

$$P(w_n|w_{n-1}) = \frac{\text{Count}(w_{n-1}, w_n)}{\text{Count}(w_{n-1})}$$

For bigrams:

$$P_{+1}(w_n|w_{n-1}) = \frac{\text{Count}(w_{n-1}, w_n) + 1}{\text{Count}(w_{n-1}) + V}$$

Add-1 Smoothed Bigram Counts

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Add-1 Smoothed Bigram Probabilities

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

$$P_{+1}(w_n|w_{n-1}) = \frac{\text{Count}(w_{n-1}, w_n) + 1}{\text{Count}(w_{n-1}) + V}$$

N-Grams in the Modern Age: InfiniGram

The use of N-Gram models peaked around 2007 for several reasons.

1. The size of N-gram tables grows exponentially with N. Moving beyond 5 or 6 – gram models was viewed as infeasible given the need to store N-gram tables.
2. Neural language models eliminated the need for complex smoothing techniques and allowed for longer contexts with the rise of recurrent neural networks.

Recently innovations have revived interest in N-gram models.

- Using Suffix trees to store text collections to generate N-gram counts on-demand
- Novel backoff method allows the use of longer variable length N-grams
- Easy interpolation with neural LMs.

N-Grams in the Modern Age: InfiniGram

- The InfiniGram model introduces a novel approach to the Markov assumption. Instead of a fixed N-gram it dynamically chooses the longest available N-gram given the available counts.

$$P_{\infty}(w_i | w_{1:i-1}) = \frac{\text{Count}(w_{i-(n-1):i})}{\text{Count}(w_{i-(n-1):i-1})}$$

$$n = \max \{n' \in [1, i] \mid \text{Count}(w_{i-(n'-1):i-1}) > 0\}$$

- This would ordinarily require an infeasibly large table for the counts. Instead, this approach constructs a suffix tree from the training corpus and retrieves counts as needed.

InfiniGram Example

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{1:i-1})$$

P(The water is so transparent that the bottom)=

$$P(\text{The}) = 0.0024$$

$$P(\text{water} | \text{The}) = 0.0008$$

$$P(\text{is} | \text{The water}) = 0.1180$$

$$P(\text{so} | \text{The water is}) = 0.0421$$

$$8.480059886589714e-12$$

$$P(\text{transparent} | \text{The water is so}) = 0.0109$$

$$P(\text{that} | \text{The water is so transparent}) = 0.6809$$

$$P(\text{the} | \text{The water is so transparent that}) = 0.2552$$

$$P(\text{bottom} | \text{The water is so transparent that the}) = 0.4694$$

Evaluating Language Models

The best evaluation for a language model is an extrinsic evaluation. How well does it work when embedded into the application you're building!

- Put model A into an application
- Evaluate the performance of the application with model A
- Put model B into the application and evaluate
- Compare performance of the application with the two models
 - *Extrinsic evaluation*
 - *A/B Testing*

Extrinsic Evaluations



Unfortunately, this can be quite time consuming and expensive to execute. It's not something you want to do unless you're pretty sure you're close to a good solution.



It's also difficult to use extrinsic evaluations to optimize for multiple use cases. That is, situations where a single language model is used to support multiple downstream applications.



An alternative, to run rapid experiments, we can evaluate language models with an *intrinsic* evaluation.



Autoregressive Generation

- We can also compare the outputs from various orders of N -gram models. Here's some output from a Shakespeare generator.

Unigram

To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have
Every enter now coverally so let

Bigram

Hill. What means, sir. I confess she? then all sorts, he is trim, captain.

Trigram

Sweet prince, Falstaff shall die. Harry of Monmouth's grave.
This shall forbid it should be branded, if renown made it empty.
Indeed the duke: and had a verv good friend.

ollow.
an?

Quadrigram

Fl.
King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;
Will you not tell me who I am?
It cannot be but so.

Intrinsic Evaluation

To see how to do this, we'll return to the probabilistic foundations for language models. Specifically...

Given some evaluation text and several alternative language models, the model that assigns a higher probability to the evaluation text is the better model.

Intrinsic Evaluation

Standard method

- Train parameters of our model on a *training set*.
- Evaluate the model on some new data: an unseen *validation or test set*.
 - A dataset which is different than our training set, but is drawn from the same source

Intrinsic Evaluation

More specifically in our setting

- The training set will be the body of text we use to gather the counts needed for our N-gram model
- The test set consists of text from the same source but not included in the training set. This test set consists of a set of sentences that we can assign probabilities to.
- The probability of a test set is then the product of the probabilities of the sentences within it.

Given multiple models, the one that assigns a higher probability to a test set is preferred.

But...

Using probabilities directly isn't practical as an evaluation measure.

- Probabilities are inherently related to the length of the sequences and the number of sentences in the test set. We'd like a measure that's independent of length.
- Sentence probabilities can get very, very, very small, very quickly. Test sets consisting of even a modest number of sentences would quickly result in numerical underflow.

Log Probabilities

To avoid these issues, we'll instead

1. Use logs of probabilities of sentences
2. Normalize those log probabilities by length
3. Treating test set (T) as a single long sequence.

Of course, if we can't compute $P(T)$ because of underflow, using logs doesn't help. So instead, we'll apply the chain rule and push the log into the probability and switch to a sum.

Entropy

$$H(T) = -\frac{1}{N} \log_2 P_m(T)$$

$$= -\frac{1}{N} \log_2 \prod_{i=1}^N P_m(w_i | w_{1:i-1})$$

$$H(T) = -\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_{1:i-1})$$

Perplexity

Perplexity has been the traditional intrinsic measure used to evaluate language models, especially in the context of speech recognition and machine translation.

The perplexity of a model is just 2 raised to the entropy!

$$PP(T) = 2^{H(T)}$$

Perplexity and Entropy

Perplexity is roughly the average number of choices that a model would have to make in guessing the next item in a sequence.

- If it is 100% certain of the next word then the perplexity is 1 (entropy of 0; since $\log(1)=0$), since there's no choice.
- If it has no information and there are M options, then the perplexity is M . So for word prediction, with a vocab of 30k, no information would correspond to guessing randomly.

Perplexity Example

Training: 38M words of WSJ Text

Testing: 1.5M words of WSJ Text

Perplexity		
Unigram	Bigram	Trigram
962	170	109

$$\frac{170 - 109}{170} * 100 = 36\%$$

Perplexity and Entropy

$$PP(T) = 2^{H(T)}$$

Entropy		
Unigram	Bigram	Trigram
9.91	7.41	6.77

Perplexity		
Unigram	Bigram	Trigram
962	170	109

Practical Issues

There are myriad practical issues that complicate comparisons among language models:

- Content issues
 - Vocabulary size, tokenization method, treatment of unknown words, sentence segmentation, markup
- Training sets
 - Training set diversity, size, origin, splits, availability
- Test sets
 - Source of test set, match with training data, test set size, availability